

Computing Fundamental

Experiment # 10

For Loop

1. Objective/s:

- For Loop
- Syntax
- Examples
- Lab Task

2. Theory:

A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

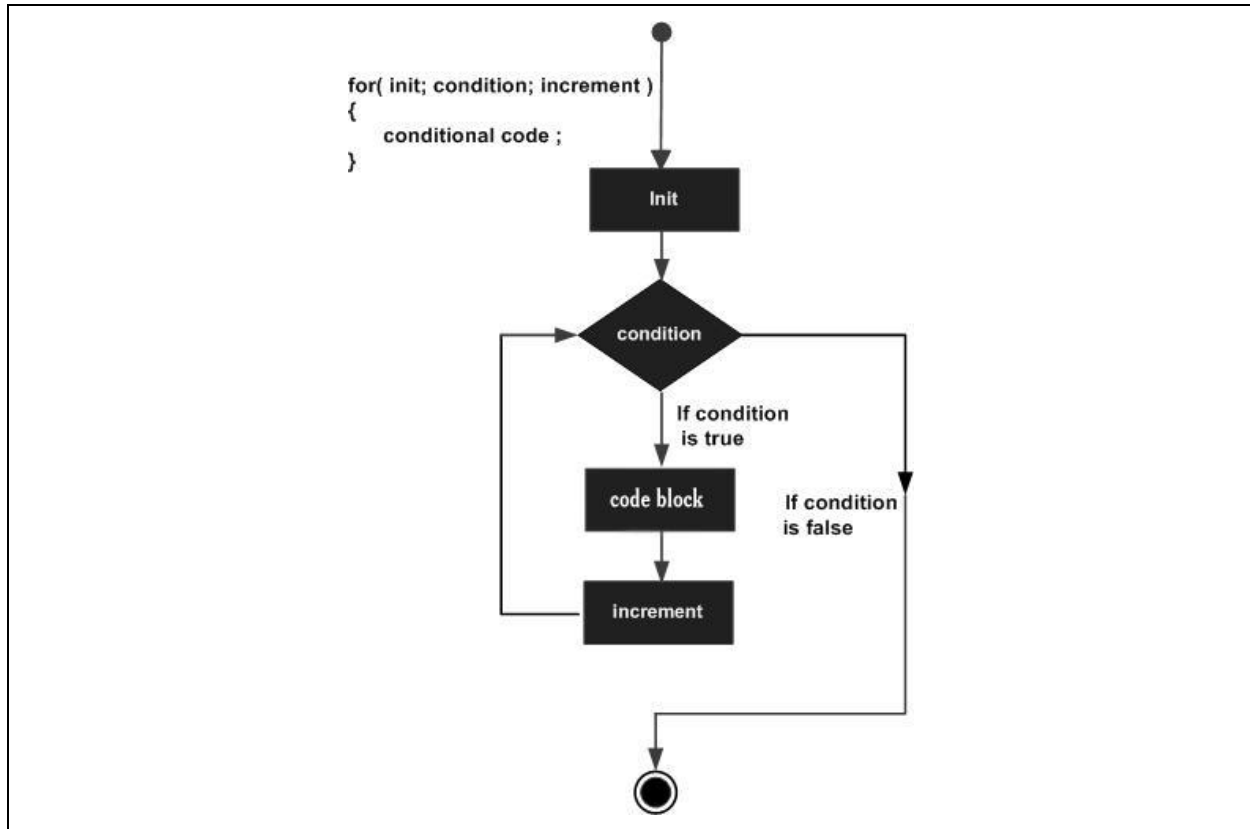
2.1. Syntax:

```
for ( init; condition; increment) {  
    Statement;  
}
```

Here is the flow of control in a 'for' loop

- The **init** step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and the flow of control jumps to the next statement just after the 'for' loop.
- After the body of the 'for' loop executes, the flow of control jumps back up to the **increment** statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the 'for' loop terminates.

2.2. Flow Diagram



2.3. Example

```
#include <stdio.h>

int main () {
    int a;

    /* for loop execution */
    for( a = 10; a < 20; a = a + 1 ){
        printf("value of a: %d\n", a);
    }
    return 0;
}
```

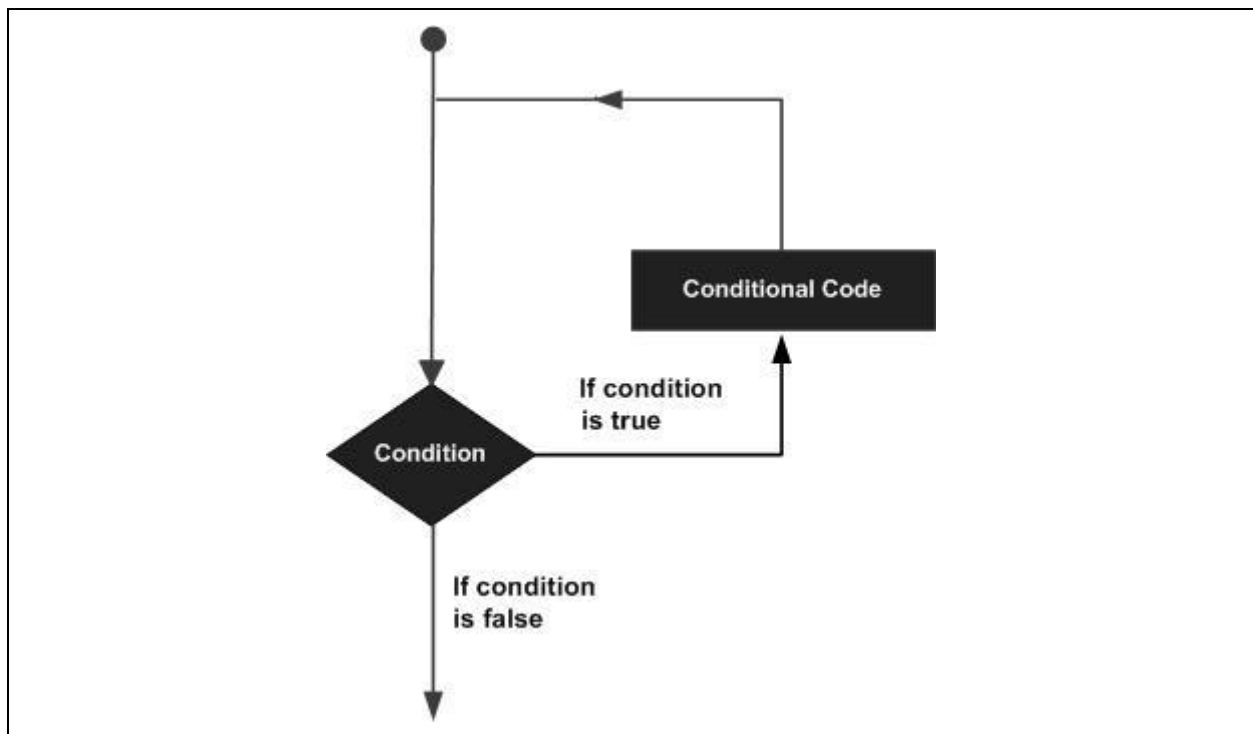
When the above code is compiled and executed, it produces the following result

```
value of a: 10  
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15  
value of a: 16  
value of a: 17  
value of a: 18  
value of a: 19
```

You may encounter situations, when a block of code needs to be executed several number of times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times. Given below is the general form of a loop statement in most of the programming languages



C programming language provides the following types of loops to handle looping requirements.

Sr.No.	Loop Type & Description
1	<u>while loop</u> Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
2	<u>for loop</u> Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
3	<u>do...while loop</u> It is more like a while statement, except that it tests the condition at the end of the loop body.
4	<u>nested loops</u> You can use one or more loops inside any other while, for, or do..while loop.

2.4. Loop Control Statements

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed.

C supports the following control statements.

Sr.No.	Control Statement & Description
1	<u>break statement</u> Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
2	<u>continue statement</u> Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

3	<u>goto statement</u> Transfers control to the labeled statement.
---	---

2.5. The Infinite Loop

A loop becomes an infinite loop if a condition never becomes false. The **for** loop is traditionally used for this purpose. Since none of the three expressions that form the 'for' loop are required, you can make an endless loop by leaving the conditional expression empty.

```
#include <stdio.h>

int main () {
    for( ; ; ) {
        printf("This loop will run forever.\n");
    }
    return 0;
}
```

When the conditional expression is absent, it is assumed to be true. You may have an initialization and increment expression, but C programmers more commonly use the `for(;;)` construct to signify an infinite loop.

3. Tasks

1. Draw following pattern using only nested for loop

```
*  
**  
***  
****  
*****  
*****  
*****
```

2. Compute factorial of a number using for loop
3. Print first 10 Fibonacci numbers using for loop